

Word Embedding (word2vec)

:label: `sec_word2vec`

Natural language is a complex system used to express meanings. In this system, words are the basic unit of the meaning. As the name implies, *word vectors* are vectors used to represent words, and can also be considered as feature vectors or representations of words. The technique of mapping words to real vectors is called *word embedding*. In recent years, word embedding has gradually become the basic knowledge of natural language processing.

One-Hot Vectors Are a Bad Choice

We used one-hot vectors to represent words (characters are words) in :numref: `sec_rnn-scratch`. Suppose that the number of different words in the dictionary (the dictionary size) is N , and each word corresponds to a different integer (index) from 0 to $N - 1$. To obtain the one-hot vector representation for any word with index i , we create a length- N vector with all 0s and set the element at position i to 1. In this way, each word is represented as a vector of length N , and it can be used directly by neural networks.

Although one-hot word vectors are easy to construct, they are usually not a good choice. A main reason is that one-hot word vectors cannot accurately express the similarity between different words, such as the *cosine similarity* that we often use. For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, their cosine similarity is the cosine of the angle between them:

$$\frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \in [-1, 1].$$

Since the cosine similarity between one-hot vectors of any two different words is 0, one-hot vectors cannot encode similarities among words.

Self-Supervised word2vec

The [word2vec](#) tool was proposed to address the above issue. It maps each word to a fixed-length vector, and these vectors can better express the similarity and analogy relationship among different words. The word2vec tool contains two models, namely *skip-gram* :cite: `Mikolov.Sutskever.Chen.ea.2013` and *continuous bag of words* (CBOW) :cite: `Mikolov.Chen.Corrado.ea.2013`. For semantically meaningful representations, their training relies on conditional probabilities that can be viewed as predicting some words using some of their surrounding words in corpora. Since

supervision comes from the data without labels, both skip-gram and continuous bag of words are self-supervised models.

In the following, we will introduce these two models and their training methods.

The Skip-Gram Model

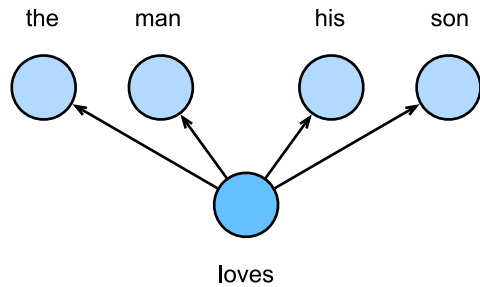
:label: subsec_skip-gram

The *skip-gram* model assumes that a word can be used to generate its surrounding words in a text sequence. Take the text sequence "the", "man", "loves", "his", "son" as an example. Let's choose "loves" as the *center word* and set the context window size to 2. As shown in :numref: fig_skip_graph, given the center word "loves", the skip-gram model considers the conditional probability for generating the *context words*: "the", "man", "his", and "son", which are no more than 2 words away from the center word:

$$P(\text{"the"}, \text{"man"}, \text{"his"}, \text{"son"} \mid \text{"loves"}).$$

Assume that the context words are independently generated given the center word (i.e., conditional independence). In this case, the above conditional probability can be rewritten as

$$P(\text{"the"} \mid \text{"loves"}) \cdot P(\text{"man"} \mid \text{"loves"}) \cdot P(\text{"his"} \mid \text{"loves"}) \cdot P(\text{"son"} \mid \text{"loves"})$$



:label: fig_skip_graph

In the skip-gram model, each word has two d -dimensional-vector representations for calculating conditional probabilities. More concretely, for any word with index i in the dictionary, denote by $\mathbf{v}_i \in \mathbb{R}^d$ and $\mathbf{u}_i \in \mathbb{R}^d$ its two vectors when used as a *center* word and a *context* word, respectively. The conditional probability of generating any context word w_o (with index o in the dictionary) given the center word w_c (with index c in the dictionary) can be modeled by a softmax operation on vector dot products:

$$P(w_o \mid w_c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)},$$

:eqlabel: eq_skip-gram-softmax

where the vocabulary index set $\mathcal{V} = \{0, 1, \dots, |\mathcal{V}| - 1\}$. Given a text sequence of length T , where the word at time step t is denoted as $w^{(t)}$. Assume that context words are independently generated given any center word. For context window size

m , the likelihood function of the skip-gram model is the probability of generating all context words given any center word:

$$\prod_{t=1}^T \prod_{-m \leq j \leq m, j \neq 0} P(w^{(t+j)} | w^{(t)}),$$

where any time step that is less than 1 or greater than T can be omitted.

Training

The skip-gram model parameters are the center word vector and context word vector for each word in the vocabulary. In training, we learn the model parameters by maximizing the likelihood function (i.e., maximum likelihood estimation). This is equivalent to minimizing the following loss function:

$$-\sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w^{(t+j)} | w^{(t)}).$$

When using stochastic gradient descent to minimize the loss, in each iteration we can randomly sample a shorter subsequence to calculate the (stochastic) gradient for this subsequence to update the model parameters. To calculate this (stochastic) gradient, we need to obtain the gradients of the log conditional probability with respect to the center word vector and the context word vector. In general, according to :eqref: eq_skip-gram-softmax the log conditional probability involving any pair of the center word w_c and the context word w_o is

$$\log P(w_o | w_c) = \mathbf{u}_o^\top \mathbf{v}_c - \log \left(\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c) \right).$$

:eqlabel: eq_skip-gram-log

Through differentiation, we can obtain its gradient with respect to the center word vector $\mathbf{v}_c$ as

$$\begin{aligned} \frac{\partial \log P(w_o | w_c)}{\partial \mathbf{v}_c} &= \mathbf{u}_o - \frac{\sum_{j \in \mathcal{V}} \exp(\mathbf{u}_j^\top \mathbf{v}_c) \mathbf{u}_j}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)} \\ &= \mathbf{u}_o - \sum_{j \in \mathcal{V}} \left(\frac{\exp(\mathbf{u}_j^\top \mathbf{v}_c)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)} \right) \mathbf{u}_j \\ &= \mathbf{u}_o - \sum_{j \in \mathcal{V}} P(w_j | w_c) \mathbf{u}_j. \end{aligned}$$

:eqlabel: eq_skip-gram-grad

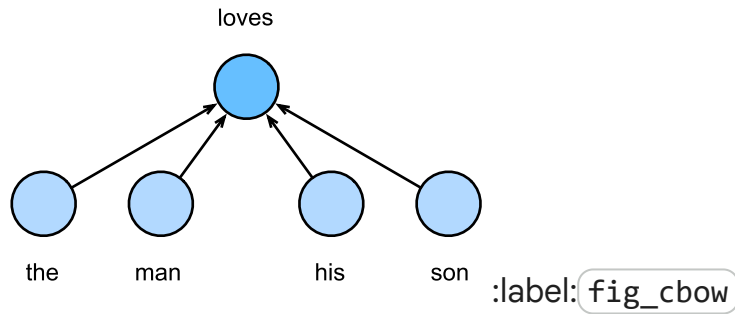
Note that the calculation in :eqref: eq_skip-gram-grad requires the conditional probabilities of all words in the dictionary with w_c as the center word. The gradients for the other word vectors can be obtained in the same way.

After training, for any word with index i in the dictionary, we obtain both word vectors $\mathbf{v}_i$ (as the center word) and $\mathbf{u}_i$ (as the context word). In natural language processing applications, the center word vectors of the skip-gram model are typically used as the word representations.

The Continuous Bag of Words (CBOW) Model

The *continuous bag of words* (CBOW) model is similar to the skip-gram model. The major difference from the skip-gram model is that the continuous bag of words model assumes that a center word is generated based on its surrounding context words in the text sequence. For example, in the same text sequence "the", "man", "loves", "his", and "son", with "loves" as the center word and the context window size being 2, the continuous bag of words model considers the conditional probability of generating the center word "loves" based on the context words "the", "man", "his" and "son" (as shown in :numref:fig_cbow), which is

$$P(\text{"loves"} \mid \text{"the"}, \text{"man"}, \text{"his"}, \text{"son"}).$$



Since there are multiple context words in the continuous bag of words model, these context word vectors are averaged in the calculation of the conditional probability. Specifically, for any word with index i in the dictionary, denote by $\mathbf{v}_i \in \mathbb{R}^d$ and $\mathbf{u}_i \in \mathbb{R}^d$ its two vectors when used as a *context* word and a *center* word (meanings are switched in the skip-gram model), respectively. The conditional probability of generating any center word w_c (with index c in the dictionary) given its surrounding context words $w_{o_1}, \dots, w_{o_{2m}}$ (with index $o_1, \dots, o_{2m}$ in the dictionary) can be modeled by

$$P(w_c \mid w_{o_1}, \dots, w_{o_{2m}}) = \frac{\exp\left(\frac{1}{2m} \mathbf{u}_c^\top (\mathbf{v}_{o_1} + \dots + \mathbf{v}_{o_{2m}})\right)}{\sum_{i \in \mathcal{V}} \exp\left(\frac{1}{2m} \mathbf{u}_i^\top (\mathbf{v}_{o_1} + \dots + \mathbf{v}_{o_{2m}})\right)}.$$

:eqlabel:fig_cbow-full

For brevity, let $\mathcal{W}_o = \{w_{o_1}, \dots, w_{o_{2m}}\}$ and $\bar{\mathbf{v}}_o = (\mathbf{v}_{o_1} + \dots + \mathbf{v}_{o_{2m}}) / (2m)$. Then :eqref:fig_cbow-full can be simplified as

$$P(w_c \mid \mathcal{W}_o) = \frac{\exp(\mathbf{u}_c^\top \bar{\mathbf{v}}_o)}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \bar{\mathbf{v}}_o)}.$$

Given a text sequence of length T , where the word at time step t is denoted as $w^{(t)}$. For context window size m , the likelihood function of the continuous bag of words model is the probability of generating all center words given their context words:

$$\prod_{t=1}^T P(w^{(t)} \mid w^{(t-m)}, \dots, w^{(t-1)}, w^{(t+1)}, \dots, w^{(t+m)}).$$

Training

Training continuous bag of words models is almost the same as training skip-gram models. The maximum likelihood estimation of the continuous bag of words model is equivalent to minimizing the following loss function:

$$-\sum_{t=1}^T \log P(w^{(t)} \mid w^{(t-m)}, \dots, w^{(t-1)}, w^{(t+1)}, \dots, w^{(t+m)}).$$

Notice that

$$\log P(w_c \mid \mathcal{W}_o) = \mathbf{u}_c^\top \bar{\mathbf{v}}_o - \log \left(\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \bar{\mathbf{v}}_o) \right).$$

Through differentiation, we can obtain its gradient with respect to any context word vector $\mathbf{v}_{o_i} (i = 1, \dots, 2m)$ as

$$\frac{\partial \log P(w_c \mid \mathcal{W}_o)}{\partial \mathbf{v}_{o_i}} = \frac{1}{2m} \left(\mathbf{u}_c - \sum_{j \in \mathcal{V}} \frac{\exp(\mathbf{u}_j^\top \bar{\mathbf{v}}_o) \mathbf{u}_j}{\sum_{i \in \mathcal{V}} \exp(\mathbf{u}_i^\top \bar{\mathbf{v}}_o)} \right) = \frac{1}{2m} \left(\mathbf{u}_c - \sum_{j \in \mathcal{V}} P(u$$

:eqlabel: eq_cbow-gradient

The gradients for the other word vectors can be obtained in the same way. Unlike the skip-gram model, the continuous bag of words model typically uses context word vectors as the word representations.

Summary

- Word vectors are vectors used to represent words, and can also be considered as feature vectors or representations of words. The technique of mapping words to real vectors is called word embedding.
- The word2vec tool contains both the skip-gram and continuous bag of words models.
- The skip-gram model assumes that a word can be used to generate its surrounding words in a text sequence; while the continuous bag of words model assumes that a center word is generated based on its surrounding context words.

Exercises

1. What is the computational complexity for calculating each gradient? What could be the issue if the dictionary size is huge?
2. Some fixed phrases in English consist of multiple words, such as "new york". How to train their word vectors? Hint: see Section 4 in the word2vec paper :cite:Mikolov.Sutskever.Chen.ea.2013.
3. Let's reflect on the word2vec design by taking the skip-gram model as an example. What is the relationship between the dot product of two word vectors in the skip-gram model and the cosine similarity? For a pair of words with similar semantics, why may the cosine similarity of their word vectors (trained by the skip-gram model) be high?

[Discussions](#)